

Incident Response Management

ERP/1 SRE Team

July 15, 2026

Abstract

This article presents a lightweight, Git-native methodology for managing incident response playbooks within the ERP/1 Kubernetes platform. By maintaining all documentation in version-controlled Markdown files, the approach achieves maximal simplicity, auditability, traceability, and seamless integration with GitOps principles.

Contents

1	Introduction	2
1.1	Core Principles from Google SRE	2
1.2	The 5 A's Framework for Excellent Playbooks	2
1.3	Key Structural Best Practices	3
1.4	Operational Best Practices	3
1.5	Common Pitfalls to Avoid	3
2	ERP/1 Playbooks	4
2.1	Detailed Playbook Template Specification	4
2.2	Architecture	4
2.3	Best Practices Implemented	5
2.4	Usage During Incidents	5
2.5	Advantages of Pure Git Approach	5
2.6	Future Enhancements	5
3	Conclusion	5

1 Introduction

In Site Reliability Engineering, playbooks are critical for reducing Mean Time To Recovery (MTTR). Google SRE practices demonstrate that well-documented playbooks can improve incident resolution times by compared to ad-hoc responses. For the ERP/1 platform, we adopted a pure git strategy ¹:

- No heavy tools or databases
- Full version history and collaboration via pull requests
- Seamless integration with ArgoCD and the existing CD pipeline
- Easy search and rendering on GitHub

1.1 Core Principles from Google SRE

- **Playbooks dramatically improve MTTR** — Google observed roughly 3x faster resolution when on-call engineers follow documented playbooks instead of “winging it.”
- **Preparation beats heroics** — Invest in up-to-date, actionable documentation + regular practice (e.g., “Wheel of Misfortune” drills).
- **Blameless culture** — Focus postmortems on systems and processes, not people.
- **Automate where possible** — But keep humans in the loop for judgment.

1.2 The 5 A’s Framework for Excellent Playbooks

A widely recommended structure for high-quality playbooks includes:

1. **Actionable** — Every step is a clear command/checklist (not prose).
2. **Accessible** — Available where responders work (Git, linked from alerts).
3. **Accurate** — Regularly reviewed and tested.
4. **Authoritative** — Single source of truth with ownership last update date.
5. **Adaptable** — Easy to update; includes decision trees for variations.

¹<https://github.com/erpuno/cd/tree/main/playbooks>

1.3 Key Structural Best Practices

- Standard Template per playbook — Title + Severity + Impact, Detection (alerts / SLIs), Immediate Actions, Diagnosis / Troubleshooting flow (with commands), Resolution steps, Escalation path, Prevention / Follow-up, Metadata.
- Keep it concise — Aim for scannable under stress. Use numbered steps, code blocks, and decision trees.
- Make it executable — Tools like Runme turn Markdown code blocks into one-click commands.
- Version control — Store in Git (Markdown) alongside code. Link to service diagrams, dashboards, and dependencies.
- Service-specific and Generic — Combine broad (e.g., “Resource Exhaustion”) and component-specific playbooks.

1.4 Operational Best Practices

- Regular reviews & testing — Quarterly or after every Sev1/2 incident. Test via chaos engineering and tabletop exercises.
- Integrate with tools — Link playbooks directly from Prometheus/Grafana alerts.
- Train & practice — On-call rotations should include playbook familiarization and simulated incidents.
- Measure & improve — Track playbook usage versus MTTR. Update based on real postmortems.
- Minimize toil — Automate repetitive steps where appropriate.
- Include architecture context — Diagrams, dependencies, recent changes, known issues.

1.5 Common Pitfalls to Avoid

- Outdated steps (leads to worse outcomes).
- Too much narrative instead of commands.
- Buried in wikis (hard to find during incidents).
- No ownership or review process.
- Ignoring non-technical aspects (communication templates, stakeholder updates).

2 ERP/1 Playbooks

2.1 Detailed Playbook Template Specification

To satisfy all requirements of the 5 A's framework and Google SRE best practices, every playbook in the repository follows this standardized template:

- **Title & Metadata** — Clear title, severity level, impact statement, owner, last reviewed date.
- **Impact & Scope** — Description of affected services/namespaces and potential business impact.
- **Detection** — Specific alerts, SLIs, or symptoms that trigger the playbook.
- **Immediate Actions** (First 5 Minutes) — High-priority steps to stabilize the situation.
- **Diagnosis** — Step-by-step troubleshooting with commands, logs, and decision trees.
- **Resolution** — Detailed remediation procedures, including rollback and verification steps.
- **Escalation Path** — Clear criteria and contacts for further escalation.
- **Prevention & Follow-up** — Long-term improvements, chaos tests, and postmortem requirements.
- **References** — Links to architecture diagrams, dashboards, related playbooks, and playbooks.

This template ensures playbooks are concise yet comprehensive, scannable under pressure, and fully executable (using code blocks). It balances generic patterns with service-specific guidance.

2.2 Architecture

The playbooks live in a dedicated `playbooks/` directory:

```
README.md
01-sev1-cluster-outage.md
02-sev2-application-degradation.md
03-sev2-resource-exhaustion.md
04-sev2-gitops-network-stateful.md
```

Each file follows a template optimized for on-call use under pressure.

2.3 Best Practices Implemented

1. **Actionable** — Every playbook contains ready-to-run `kubectl`, `argocd`.
2. **Versioned** — Git tracks every change with clear commit messages.
3. **Searchable** — GitHub search + static site.
4. **Executable** — Compatible for one-click execution.
5. **Reviewed** — Changes via PRs with SRE approval.

2.4 Usage During Incidents

1. Open the relevant playbook directly from GitHub or local clone.
2. Follow numbered steps.
3. Document actions in the incident record.
4. After resolution, update the playbook if new lessons were learned.

2.5 Advantages of Pure Git Approach

- Zero operational overhead
- Works offline
- Perfect audit trail for compliance (NIST)
- Natural fit for GitOps culture
- Easy to extend with automation (e.g., linking from Prometheus alerts)

2.6 Future Enhancements

- Runme integration for executable blocks
- Automated testing of playbook steps
- AI-assisted playbook generation

3 Conclusion

Managing playbooks in pure Git represents the SRE ideal: simplicity, reliability, and developer-friendly workflows. This approach keeps our focus on reliability rather than tool administration.